

EasyBank Database

Design Report

Zane Grose

3/6/24

MSA106

Table of Contents

.....0
Document Objectives:.....2
Requirements, Assumptions, and Constraints: 2
Requirements: 2
Assumptions:.....3
Constraints: 3
Database Diagrams:.....4
Future Functionalities: 7
Predicting a user's savings: 7
Additional security login method for online:.....8
Process of Design: 8
Screenshots & Code Output: 8
SQL Queries:.....8
I. Creation 8
II. Insertion 10
III. Updating 12
SQL Query Output: 13
Future Improvements: 16
Database Script: 17
Database Setup:.....17
Data Analysis: 18
Reflection:.....18
Bibliography 18

Document Objectives:

To provide sufficient information on the process of creating a Database Structure for the EasyBank scenario.

- This document will include a diagram that demonstrated the database design, such as an ER Model, ER Diagram and Use Case Diagram.
- Suggestions on how to solve the future functionalities such as predicting a person's savings or adding an additional security method for online login.
- The process of design that I went through to create this document, design the database and what order I did all my processes in.
- Screenshots of the code queries for creation, inserting and updating information within the tables described in the diagrams.
- A section on future improvements and based off the proposed database design, what could be improved upon.
- Reflection on the project, outlining what I think went well or bad in the process of this project.

Requirements, Assumptions, and Constraints:

Requirements:

- Able to store customer information.
- Able to create a bank account.
- Dynamically change the interest rate depending on the type of bank account.
- Customers of any age can create an account.
- Have an online banking feature to show the customer accounts transaction history. (Example query)
- A query to show customers to offer loans to (over 18 and making over a certain salary)
- A query to compare the number of accounts to branches.
- A query to calculate the difference between people under 30 and over 50 years old.
- Description on how to predict a person's savings.
- Description on how to have an additional security method for online login.

Assumptions:

As EasyBank has multiple branches and is highly likely to scale up, it is assumed that they will have to migrate to a NewSQL database in the future.

No one outside of EasyCountry will be able to register for an account to EasyBank, as well as no transactions will be able to be made between established banks in EasyCountry, this will restrict the transactions between EasyBank accounts.

Customers under 18 will require a guardian to go into the bank with them to provide contact information. It is also assumed that a customer must go into a branch to setup an account, but they can register for an online login before going into the branch.

When a customer registers at the branch, the staff member will have to create an online login (or help the customer through the process) for the customer if they do not already have an account, otherwise, the staff member will utilize the customers currently stored information to create an account for the customer.

Finally, it's assumed that an Account can only be related to one Branch, this helps EasyBank manage Accounts that have been set up in their system.

Constraints:

As only have experience with MySQL, I will be limited to picking a SQL database, NoSQL and NewSQL will not be considered in this project. NewSQL will not be implemented till a MySQL database is implemented to structure the data appropriately for migration to a NewSQL database, in the future.

EasyBank transactions will only be available to other EasyBank accounts, this is due to EasyBank not being certified by EasyGovernment, not having a national bank number.

Database Diagrams:

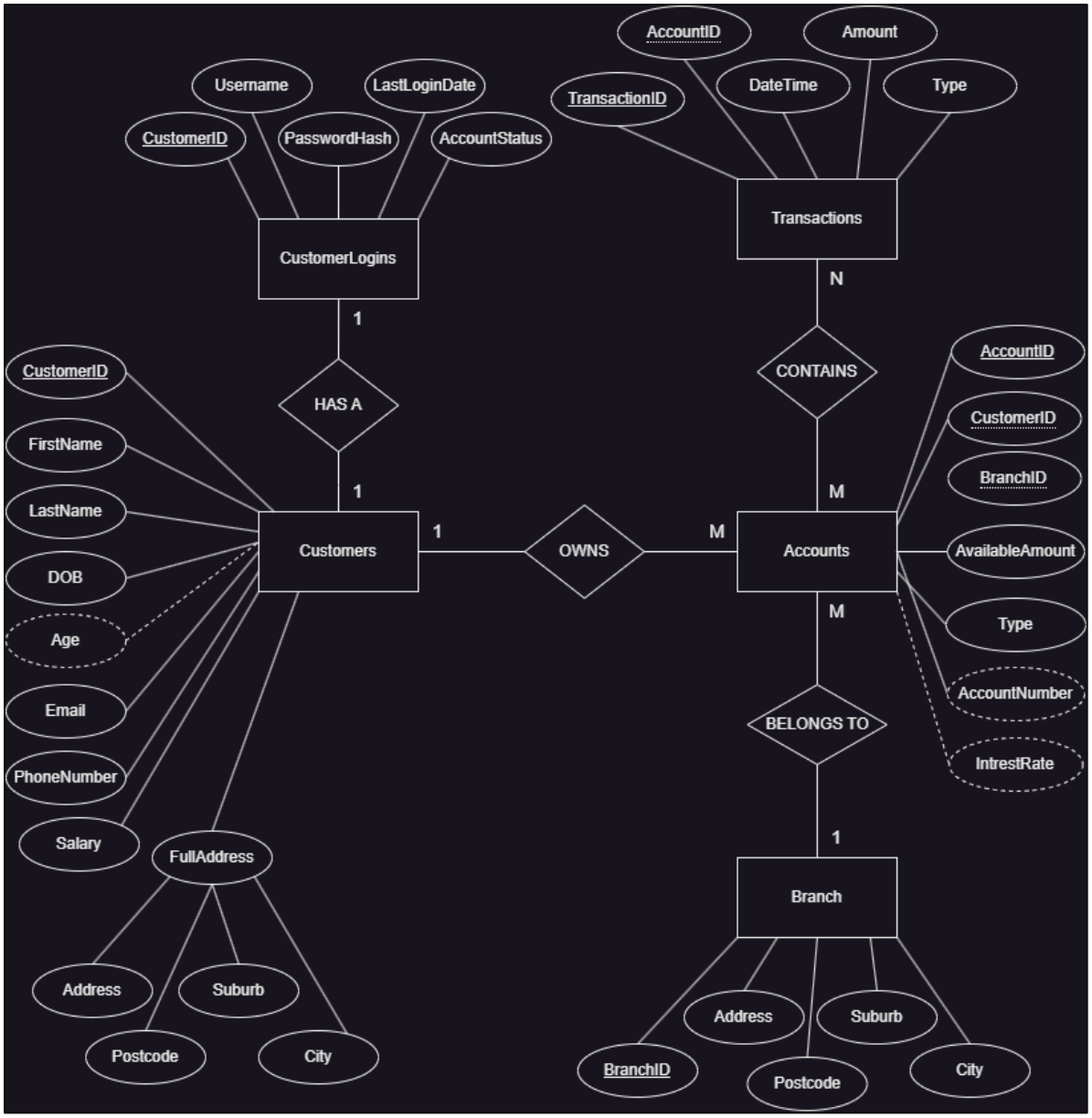


Figure 1: Entity-Relationship Model of proposed EasyBank database structure.

Many customers having the same transactions was considered, and this was the factor that determined the many-to-many relationship between accounts and transactions. Derived variables are created within the database setup and set to null initially, after a customer or account is created in the database, it would run the queries to generate the age, account number and interest rate.

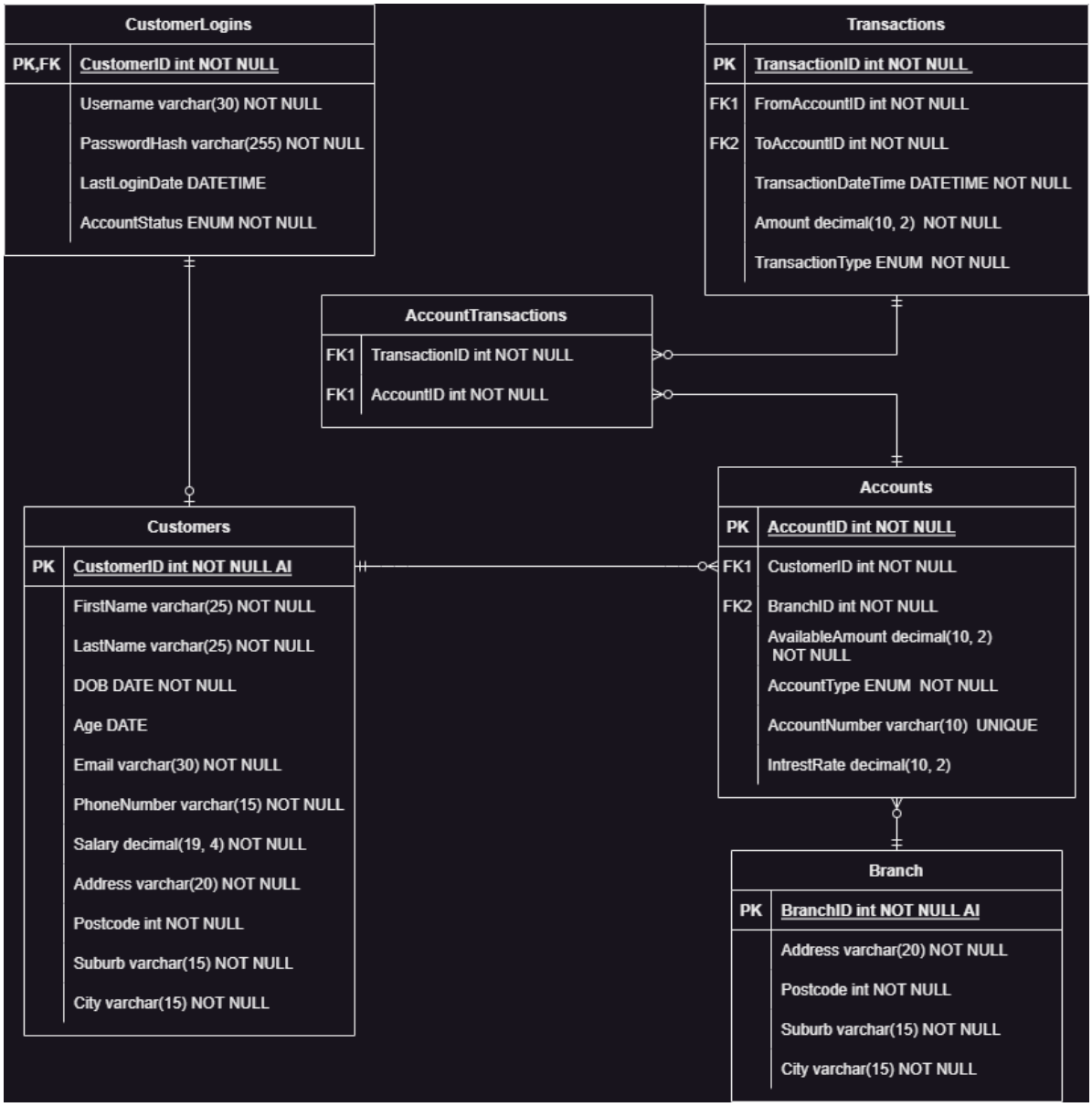


Figure 2: Entity-Relationship Model of proposed EasyBank database structure.

Note in Figure 2, that the derived variables from Figure 1 are designated in such a way that they can be null. This allows us to create a SQL query to create each derived attributes values with existing information after initial creation of a record.

The relationship between the accounts and transactions has been upgraded to a junction table in this ERD. This is a common approach to many-to-many relationships in database design. It allows modularity and security in the database by keeping a single transaction record and accessing it through the junction table, reducing redundancy. It also allows the transactions table to be access restricted, and only allowing the customer to get transaction information only when their ID is equal to the AccountTransactions foreign key restraint.

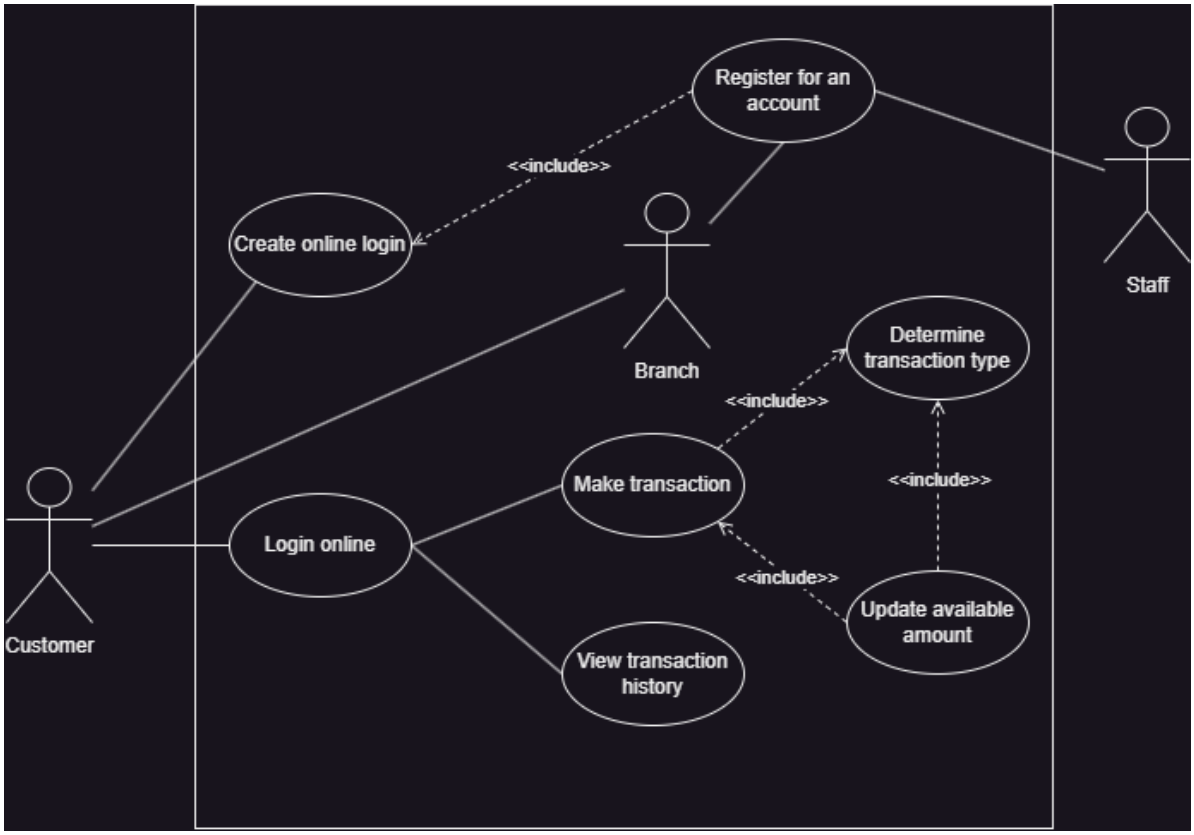


Figure 3: Use Case Diagram of a user's interaction with the database and external entities.

This use case is a representation of the customers interaction with the EasyBank system. The branch is a part of the system and an actor, which is why it is represented as inside the system, but also as an actor.

The user can create an online login, login online, make a transaction, and view their transaction history. To open a usable bank account though, they must physically visit a branch and interact with a staff member. Creating a bank account requires creating an online login.

Customers viewing their transaction history will receive all Transaction records that have their corresponding ID within the junction table in the ERD. Making a transaction requires the system to determine the transaction type, conversely, the system updating the available amount requires itself to also determine how to calculate the transaction made.

Future Functionalities:

Predicting a user's savings:

There are two ways to implement this and a couple of things to consider when implementing this functionality.

If predicting using only savings type accounts deposits, you would utilize the AccountTransactions table for each customer, checking the transaction type, and the amount of money moved in the transaction. Using that information, you can determine how much money was deposited into the account each week or each year (Ex. $50 p/week, $50 * 52 weeks p/year = $2,600), then that result is the average amount that customer will put into their savings account each year. Note that the value of $50 per week is a static amount, this will be different each week/month for each customer. This adds an extra step, which will require averaging the amount they put into their savings for each week, over a yearly period – which will result in the average amount saved yearly.

```
# Query example of data needed to predict someones savings
SELECT t.FromAccountID, t.ToAccountID, t.TransactionType, t.Amount, a.AccountType
FROM transactions t
JOIN accounts a
ON t.FromAccountID = a.AccountID OR t.ToAccountID = a.AccountID
WHERE a.AccountType = 'Savings' AND (t.TransactionType = 'Deposit' OR  t.TransactionType = 'Transfer');
```

Figure 4: Example Query for data needed to predict a person's savings.

	FromAccountID	ToAccountID	TransactionType	Amount	AccountType
▶	108	109	Transfer	100.00	Savings
	112	109	Deposit	250.00	Savings
	112	111	Transfer	1000.00	Savings

Figure 5: Output of Figure 4.

The second option is to implement a machine learning algorithm using a decision tree model. This has been proven to be efficient, "The decision tree model was found to be the best prediction model to predict savings adequacy as it performed well across all evaluation metrics." This process considers the 'mental accounting' theory, which does have predictive power on savings adequacy. (Alam, Chen, & Mansor, 2022). This would be more accurate and efficient than manually calculating someone's savings, rather than only using the real data being put into the savings account. EasyBank would have to consider a branch to oversee savings-related processes, such as interacting with the customer to gain the savings goals, saving biases, and mental processes from the customer, which would be utilized to predict their savings adequacy.

Additional security login method for online:

There are many different additional security login methods, but the first one that comes to mind is an OTP (One-Time Password). This would generate a temporary code and send it to the customers phone number, email, or authentication app after the user successfully enters their login details. The website would then ask the customer to enter that number, check if the entered number is equal to the generated number temporarily stored in the system. If wanting to implement an OTP algorithm within EasyBank's current system, an OTP algorithm will be developed, then that will be passed into a TOTP algorithm that increases security. (M'Raihi, et al., 2011)

Process of Design:

The first thing that was done was understanding the requirements of EasyBank and listing them out at the start of this document. Then creating the base design for the database which involved designing an ER Model, then creating an ERD off that model. Finally, a use case diagram was developed to outline the processes of the users, staff, branch system and their interactions with the system. The database and tables were then created, populating the tables with dummy data. This allowed me to debug the code and ensure that the queries for inserting data were working as expected. After this, I did a lot of re-iteration for each table in the database which included collecting requirements, designing, development, testing, review, and iteration that informed decisions to update queries, design documents and this report.

Screenshots & Code Output:

SQL Queries:

I. Creation

EasyBank Database:

```
CREATE DATABASE EasyBank;

USE EasyBank;
```

Figure 6: Creating and using the database for subsequent table queries.

Customers Table:

```
CREATE TABLE IF NOT EXISTS customers (
    CustomerID int NOT NULL auto_increment,
    FirstName varchar(25) NOT NULL,
    LastName varchar(25) NOT NULL,
    DOB date NOT NULL,
    Email varchar(30),
    PhoneNumber varchar(15) NOT NULL,
    Salary decimal(19, 4) NOT NULL,
    Address varchar(20) NOT NULL,
    Postcode int NOT NULL,
    Suburb varchar(15) NOT NULL,
    City varchar(15) NOT NULL,
    Age int,
    PRIMARY KEY(CustomerID)
);
```

Figure 8: Query for Customers Table creation.

CustomerLogins Table:

```
CREATE TABLE IF NOT EXISTS customerlogins(
    CustomerID int NOT NULL,
    Username varchar(30) NOT NULL,
    PasswordHash varchar(255) NOT NULL,
    LastLoginDate DATETIME NOT NULL,
    AccountStatus ENUM('Locked', 'Pending', 'Active') NOT NULL,
    PRIMARY KEY(CustomerID),
    FOREIGN KEY (CustomerID) REFERENCES customers(CustomerID)
);
```

Figure 7: Query for CustomerLogins table creation.

Branch Table:

```
CREATE TABLE IF NOT EXISTS branch(
    BranchID int NOT NULL AUTO_INCREMENT,
    Address varchar(20) NOT NULL,
    Postcode int NOT NULL,
    Suburb varchar(15) NOT NULL,
    City varchar(15) NOT NULL,
    PRIMARY KEY (BranchID)
) AUTO_INCREMENT = 01;
```

Figure 10: Query for Branch table creation.

Accounts Table:

```
CREATE TABLE IF NOT EXISTS accounts(
    AccountID int NOT NULL AUTO_INCREMENT,
    CustomerID int NOT NULL,
    BranchID int NOT NULL,
    AvailableAmount decimal(10, 2) NOT NULL,
    AccountType ENUM('Check', 'Savings', 'Loan') NOT NULL,
    AccountNumber varchar(30) UNIQUE,
    IntrestRate decimal(10, 2),
    FOREIGN KEY(CustomerID) REFERENCES customers(CustomerID),
    FOREIGN KEY(BranchID) REFERENCES branch(BranchID),
    PRIMARY KEY(AccountID)
) AUTO_INCREMENT = 100;
```

Figure 9: Query for Accounts table creation.

Transactions Table:

```
CREATE TABLE IF NOT EXISTS transactions(
    TransactionID int NOT NULL AUTO_INCREMENT,
    FromAccountID int NOT NULL,
    ToAccountID int NOT NULL,
    TransactionDateTime DATETIME NOT NULL,
    Amount decimal(10, 2) NOT NULL,
    TransactionType ENUM('Deposit', 'Transfer', 'Payment'),
    FOREIGN KEY(FromAccountID) REFERENCES accounts(AccountID),
    FOREIGN KEY(ToAccountID) REFERENCES accounts(AccountID),
    PRIMARY KEY(TransactionID)
);
```

Figure 12: Query for Transactions table creation.

AccountTransactions Table:

```
CREATE TABLE IF NOT EXISTS accounttransactions (
    TransactionID int NOT NULL,
    ToAccountID int NOT NULL,
    FromAccountID int NOT NULL,
    FOREIGN KEY(TransactionID) REFERENCES transactions(TransactionID),
    FOREIGN KEY(ToAccountID) REFERENCES accounts(AccountID),
    FOREIGN KEY(FromAccountID) REFERENCES accounts(AccountID),
    PRIMARY KEY (TransactionID, ToAccountID, FromAccountID)
);
```

Figure 11: Query for AccountTransactions table creation.

II. Insertion

Customers Table:

```
INSERT INTO customers(FirstName, LastName, DOB, Email, PhoneNumber, Salary, Address, Postcode, Suburb, City) VALUES
("Billy", "Strongson", "1990-05-15", "billsson@gmail.com", "02954827527", 100000.00, "5 Strongs Street", 0156, "EasyCenter", "EasyCity"),
("Parvati", "Jha", "1980-12-08", "parvjh@outlook.com", "02752820591", 90000.00, "20 Main Street", 0156, "EasyCenter", "EasyCity"),
("Abdul", "Jha", "2005-01-08", "abdjh@gmail.com", "0225257291", 40000.00, "20 Main Street", 0156, "EasyCenter", "EasyCity"),
("Lilly","Honey","1995-10-01","lhoney54@gmail.com","02167382956", 115000.00, "93 Cobble Avenue", 0150, "EasyRural", "EasyCity"),
("Dylan", "Simpleman", "1995-03-24", "dylsimp@gmail.com", "02254322020", 75000.00, "93 Simple Lane", 0150, "EasyRural", "EasyCity"),
("John","Small","1984-08-26","JSM4LL@gmail.com","023273023455", 75000.00, "104 Farmer Lane", 0140, "SouthRural", "SouthCity"),
("Dylan","Small","1980-02-20","dylanlight@outlook.com","022279073455", 75000.00, "104 Farmer Lane", 0140, "SouthRural", "SouthCity"),
("Alissa","Small","2010-06-08","dylanlight@outlook.com","022279073455", 00.00, "104 Farmer Lane", 0140, "SouthRural", "SouthCity"),
("Boss","Original","1950-02-08","bossO@outlook.com","022805534252", 300000.00, "1 Main Street", 0156, "EasyCenter", "EasyCity");
```

Figure 13: Query for Customers table record insertion.

The customer would enter their data when in the process of registering an online login.

CustomerLogins Table:

```
INSERT INTO customerlogins(CustomerID, Username, PasswordHash, LastLoginDate, AccountStatus)
SELECT CustomerID, Email, 'hashed_password_here', NOW(), 'Pending'
FROM customers;
```

Figure 14: Query for CustomerLogins record insertion.

This customer login data will also be entered in the process of registering and online login.

Branch Table:

```
INSERT INTO branch(Address, Postcode, Suburb, City) VALUES
("2 Main Street", 0156, "EasyCenter", "EasyCity"),
("10 Cobble Street", 0150, "EasyRural", "EasyCity"),
("102 Longs Road", 0145, "SouthCenter", "SouthCity");
```

Figure 15: Query for Branch table record insertion.

A system administrator would enter the branch information.

Accounts Table:

```
INSERT INTO accounts(CustomerID, BranchID, AvailableAmount, AccountType) VALUES
(1, 1, 5000.00, 'Check'),
(2, 1, 50.00, 'Savings'),
(2, 1, 550.00, 'Check'),
(3, 2, 700.00, 'Check'),
(4, 2, 100000.00, 'Check'),
(5, 3, 10.00, 'Check'),
(5, 3, -5000.00, 'Loan'),
(6, 3, 10.00, 'Check'),
(7, 3, 350.00, 'Check'),
(7, 3, 1565.00, 'Savings'),
(8, 3, 00.00, 'Check'),
(9, 1, 50000.00, 'Savings'),
(9, 1, 200000.00, 'Check');
```

Figure 16: Query for Accounts table record insertion.

The accounts table would be created by a staff member within a branch. This is for security purposes.

Transactions Table:

```
INSERT INTO transactions(FromAccountID, ToAccountID, Amount, TransactionType, TransactionDateTime) VALUES
(102, 103, 50.00, 'Transfer', NOW()),
(112, 100, 150.00, 'Deposit', NOW()),
(112, 102, 250.00, 'Deposit', NOW()),
(112, 108, 50.00, 'Deposit', NOW()),
(105, 106, 10.00, 'Payment', NOW()),
(108, 109, 100.00, 'Transfer', NOW()),
(112, 111, 1000.00, 'Transfer', NOW()),
(112, 109, 250.00, 'Deposit', NOW());
```

Figure 17: Query for Transactions table record insertion.

A transaction record will be made when a customer makes a transaction. The internal code will manage the foreign keys in the table with an input from the user of AccountNumber.

AccountTransactions Table:

```
INSERT IGNORE INTO accounttransactions(TransactionID, ToAccountID, FromAccountID)
SELECT t.TransactionID, t.ToAccountID, t.FromAccountID
FROM transactions t
JOIN accounts a
ON t.FromAccountID = a.AccountID OR t.ToAccountID = a.AccountID;
```

Figure 18: Query for AccountTransactions record insertion.

This query inserts the transaction data from transactions. This table is then used to return transaction information to the customer, while logged into an account. Ignore keyword is included here, as any duplicates are not needed. Each account will access the record in the AccountTransactions table using the stored already transaction record in the Transactions table.

III. Updating

These update queries must be called before outputting the data from the tables and after creating the relevant tables.

Derived Age column in Customers Table:

```
UPDATE customers
SET age = TIMESTAMPDIFF(YEAR, DOB, CURDATE())
WHERE CustomerID >= 0;
```

Figure 19: Query to populate the Age column in Customers table using the given Date of Birth.

Derived AccountNumber in Accounts Table:

```
UPDATE accounts SET AccountNumber = CONCAT(
BranchID, (FLOOR(RAND() * 9000000) + 1000000)
) WHERE AccountID >= 100;
```

Figure 20: Query to populate the Accounts table AccountNumber column, concatenating the Branch ID and a random 7-digit number.

Derived InterestRate in Accounts Table:

```
UPDATE accounts SET IntrestRate =
CASE AccountType
    WHEN 'Check' THEN 0.95
    WHEN 'Savings' THEN 3.00
    WHEN 'Loan' THEN 2.50
END
WHERE AccountID >= 100;
SELECT * FROM accounts;
```

Figure 21: Query to populate the Accounts table Interest Rate column, determining interest rate depending on the type of account.

SQL Query Output:

Customer Records:

```
SELECT * FROM customers;
```

Figure 22: Selecting all record from Customers table.

CustomerID	FirstName	LastName	DOB	Email	PhoneNumber	Salary	Address	Postcode	Suburb	City	Age
1	Billy	Strongson	1990-05-15	billsson@gmail.com	02954827527	100000.0000	5 Strongs Street	156	EasyCenter	EasyCity	33
2	Parvati	Jha	1980-12-08	parvjh@outlook.com	02752820591	90000.0000	20 Main Street	156	EasyCenter	EasyCity	43
3	Abdul	Jha	2005-01-08	abdjh@gmail.com	0225257291	40000.0000	20 Main Street	156	EasyCenter	EasyCity	19
4	Lilly	Honey	1995-10-01	lhoney54@gmail.com	02167382956	115000.0000	93 Cobble Avenue	150	EasyRural	EasyCity	28
5	Dylan	Simpleman	1995-03-24	dylsimp@gmail.com	02254322020	75000.0000	93 Simple Lane	150	EasyRural	EasyCity	29
6	John	Small	1984-08-26	JSM4LL@gmail.com	023273023455	75000.0000	104 Farmer Lane	140	SouthRural	SouthCity	39
7	Dylan	Small	1980-02-20	dylanlight@outlook.com	022279073455	75000.0000	104 Farmer Lane	140	SouthRural	SouthCity	44
8	Alissa	Small	2010-06-08	dylanlight@outlook.com	022279073455	0.0000	104 Farmer Lane	140	SouthRural	SouthCity	13
9	Boss	Original	1950-02-08	bossO@outlook.com	022805534252	300000.0000	1 Main Street	156	EasyCenter	EasyCity	74
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 23: Output of Figure 22

Customers 18 or older and making over 75000.00 annually:

```
# Query to select the list of people to offer loans to
SELECT CustomerID, FirstName, LastName, Age, Salary FROM customers
WHERE Age >= 18 AND Salary >= 75000.00;
```

Figure 24: Selecting customers over 18 with a salary over $75,000 annually.

CustomerID	FirstName	LastName	Age	Salary
1	Billy	Strongson	33	100000.0000
2	Parvati	Jha	43	90000.0000
4	Lilly	Honey	28	115000.0000
5	Dylan	Simpleman	29	75000.0000
6	John	Small	39	75000.0000
7	Dylan	Small	44	75000.0000
9	Boss	Original	74	300000.0000
NULL	NULL	NULL	NULL	NULL

Figure 25: Output of Figure 24

Average Salary difference between people under 30 and people over 50:

```
# Select the average salary difference between people under 30 and over 50
SELECT AVG(CASE WHEN Age < 30 THEN Salary ELSE NULL END) AS AverageSalaryUnder30,
       AVG(CASE WHEN Age > 50 THEN Salary ELSE NULL END) AS AverageSalaryOver50,
       AVG(CASE WHEN Age < 30 THEN Salary ELSE NULL END) - AVG(CASE WHEN Age > 50 THEN Salary ELSE NULL END) AS AverageSalaryDifference
       FROM customers;
```

Figure 26: Selecting the average salary of customers under 30, and over 50, then calculating the difference between them.

AverageSalaryUnder30	AverageSalaryOver50	AverageSalaryDifference
57500.00000000	300000.00000000	-242500.00000000

Figure 27: Output of Figure 26

CustomerLogins records:

```
SELECT * FROM customerlogins;
```

Figure 28: Selecting all records from CustomerLogins table.

CustomerID	Username	PasswordHash	LastLoginDate	AccountStatus
1	billsson@gmail.com	hashed_password_here	2024-04-08 22:24:41	Pending
2	parvjh@outlook.com	hashed_password_here	2024-04-08 22:24:41	Pending
3	abdjh@gmail.com	hashed_password_here	2024-04-08 22:24:41	Pending
4	lhoney54@gmail.com	hashed_password_here	2024-04-08 22:24:41	Pending
5	dylsimp@gmail.com	hashed_password_here	2024-04-08 22:24:41	Pending
6	JSM4LL@gmail.com	hashed_password_here	2024-04-08 22:24:41	Pending
7	dylanlight@outlook.com	hashed_password_here	2024-04-08 22:24:41	Pending
8	dylanlight@outlook.com	hashed_password_here	2024-04-08 22:24:41	Pending
9	bossO@outlook.com	hashed_password_here	2024-04-08 22:24:41	Pending
NULL	NULL	NULL	NULL	NULL

Figure 29: Output of Figure 28

Branch records:

```
SELECT * FROM branch;
```

Figure 30: Selecting all records from the Branch table.

BranchID	Address	Postcode	Suburb	City
1	2 Main Street	156	EasyCenter	EasyCity
2	10 Cobble Street	150	EasyRural	EasyCity
3	102 Longs Road	145	SouthCenter	SouthCity
NULL	NULL	NULL	NULL	NULL

Figure 31: Output of Figure 30

Accounts records:

```
SELECT * FROM accounts;
```

Figure 32: Selecting all records from the Accounts table.

AccountID	CustomerID	BranchID	AvailableAmount	AccountType	AccountNumber	IntrestRate
100	1	1	5000.00	Check	11192017	0.95
101	2	1	50.00	Savings	12714829	3.00
102	2	1	550.00	Check	18998093	0.95
103	3	2	700.00	Check	28845982	0.95
104	4	2	100000.00	Check	27235629	0.95
105	5	3	10.00	Check	38640200	0.95
106	5	3	-5000.00	Loan	32494112	2.50
107	6	3	10.00	Check	33549963	0.95
108	7	3	350.00	Check	39267479	0.95
109	7	3	1565.00	Savings	37687507	3.00
110	8	3	0.00	Check	39635100	0.95
111	9	1	50000.00	Savings	16112980	3.00
112	9	1	200000.00	Check	19659601	0.95
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 33: Output of Figure 32

Comparing number of Accounts by Branch:

```
# Compare the number of accounts by branches
SELECT BranchID, COUNT(AccountID) AS NumberOfAccounts
FROM accounts
GROUP BY BranchID;
```

Figure 34: Selecting the number of accounts by Branch ID.

BranchID	NumberOfAccounts
1	5
2	2
3	6

Figure 35: Output of Figure 34

Transactions records:

```
SELECT * FROM transactions;
```

Figure 36: Selecting all records from the Transactions table.

TransactionID	FromAccountID	ToAccountID	TransactionDateTime	Amount	TransactionType
1	102	103	2024-04-09 12:21:31	50.00	Transfer
2	112	100	2024-04-09 12:21:31	150.00	Deposit
3	112	102	2024-04-09 12:21:31	250.00	Deposit
4	112	108	2024-04-09 12:21:31	50.00	Deposit
5	105	106	2024-04-09 12:21:31	10.00	Payment
6	108	109	2024-04-09 12:21:31	100.00	Transfer
7	112	111	2024-04-09 12:21:31	1000.00	Transfer
8	112	109	2024-04-09 12:21:31	250.00	Deposit
NULL	NULL	NULL	NULL	NULL	NULL

Figure 37: Output of Figure 36.

AccountTransactions records:

```
SELECT * FROM accounttransactions;
```

Figure 38: Selecting all records from AccountTransactions table.

TransactionID	ToAccountID	FromAccountID
2	100	112
3	102	112
1	103	102
5	106	105
4	108	112
6	109	108
8	109	112
7	111	112
NULL	NULL	NULL

Figure 39: Output of Figure 38.

Select transactions for a certain AccountID example (transaction history):

```
SELECT acct.TransactionID, acct.ToAccountID, acct.FromAccountID, t.Amount, t.TransactionDateTime, t.TransactionType
FROM accounttransactions acct
JOIN transactions t ON acct.TransactionID = t.TransactionID
WHERE acct.FromAccountID = 112 OR acct.ToAccountID = 112;
```

Figure 40: Selecting transactions based on an Account ID

TransactionID	ToAccountID	FromAccountID	Amount	TransactionDateTime	TransactionType
3	100	112	150.00	2024-04-09 23:34:18	Deposit
4	102	112	250.00	2024-04-09 23:34:18	Deposit
5	108	112	50.00	2024-04-09 23:34:18	Deposit
9	109	112	250.00	2024-04-09 23:34:18	Deposit
8	111	112	1000.00	2024-04-09 23:34:18	Transfer

Figure 41: Output of Figure 40

Future Improvements:

Integrate compatibility with other banks that are established within EasyCountry. This would involve registering a Bank ID number, with EasyGovernment, that would be statically stored and concatenated like so, Bank ID + Branch + Account ID + Account Type. This would result in a unique identifying number that can be used to create transactions internally within the EasyBank server. Currently, Bank ID is omitted from the account number identifier, so it is given that the customers can only perform transactions between EasyBank accounts. The Account Type will also not be concatenated in this project, as it requires creating another table, which is out of the scope of the given diagrams.

A Customer ID reference in the Customers table to a specific guardian could be added as an optional field for Customers under 18, then that under 18 customers contact information could be filled using the guardians contact information. This would be necessary for issues regarding account management.

Database Script:

Database Setup:

To setup the database for testing, I have included a .SQL script file that creates all the database and table information, then populates those tables with dummy data. This requires MySQL workbench to utilize.



Figure 42: SQL script file to populate a SQL Server database with the Figure examples.

Opening the .SQL script file from this document will open a new tab with the plain text code. If opening directly from the file, it should open in MySQL workbench (if installed). If not, open MySQL workbench, connect to a database, then paste the plain text into the script window.

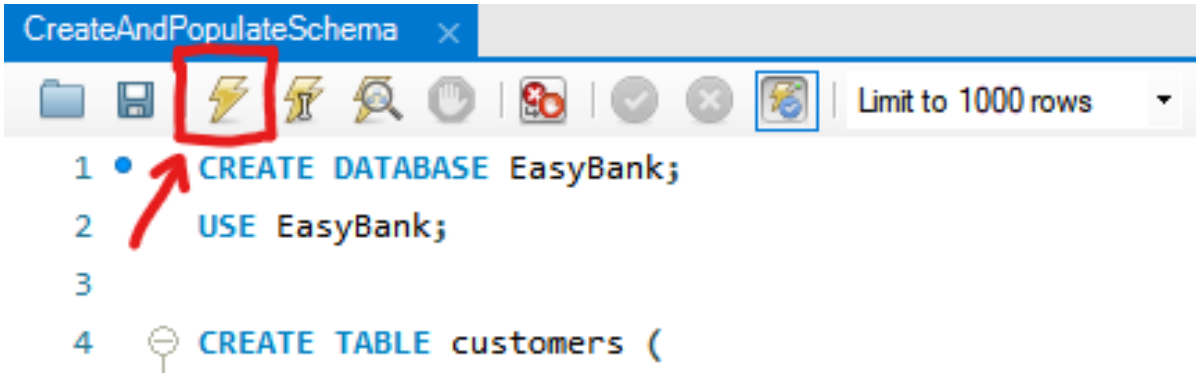
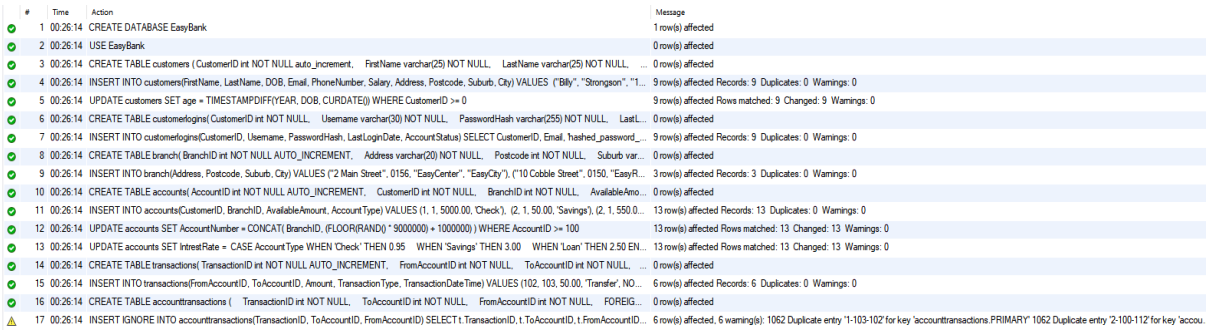


Figure 43: Button to run the entire loaded .SQL script.

After loading the query script, if you press this button above the script window, it will run the entire script and will create the database, tables, and populate the tables with data. If you haven't created the script yet and have the plain text code, then you can just copy and paste into the window below this button.

Data Analysis:



#	Time	Action	Message
1	00:26:14	CREATE DATABASE EasyBank	1 row(s) affected
2	00:26:14	USE EasyBank	0 row(s) affected
3	00:26:14	CREATE TABLE customers (CustomerID int NOT NULL auto_increment, FirstName varchar(25) NOT NULL, LastName varchar(25) NOT NULL, ...	0 row(s) affected
4	00:26:14	INSERT INTO customers(FirstName, LastName, DOB, Email, PhoneNumber, Salary, Address, Postcode, Suburb, City) VALUES ("Billy", "Strongson", "1...	9 row(s) affected Records: 9 Duplicates: 0 Warnings: 0
5	00:26:14	UPDATE customers SET age = TIMESTAMPDIFF(YEAR, DOB, CURDATE()) WHERE CustomerID >= 0	9 row(s) affected Rows matched: 9 Changed: 9 Warnings: 0
6	00:26:14	CREATE TABLE customerlogins(CustomerID int NOT NULL, Username varchar(30) NOT NULL, PasswordHash varchar(255) NOT NULL, LastL...	0 row(s) affected
7	00:26:14	INSERT INTO customerlogins(CustomerID, Username, PasswordHash, LastLoginDate, AccountStatus) SELECT CustomerID, Email, 'hashed_password_...	9 row(s) affected Records: 9 Duplicates: 0 Warnings: 0
8	00:26:14	CREATE TABLE branch(BranchID int NOT NULL AUTO_INCREMENT, Address varchar(20) NOT NULL, Postcode int NOT NULL, Suburb var...	0 row(s) affected
9	00:26:14	INSERT INTO branch(Address, Postcode, Suburb, City) VALUES ("2 Main Street", 0156, "EasyCenter", "EasyCity"), ("10 Cobble Street", 0150, "EasyR...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
10	00:26:14	CREATE TABLE accounts(AccountID int NOT NULL AUTO_INCREMENT, CustomerID int NOT NULL, BranchID int NOT NULL, AvailableAmo...	0 row(s) affected
11	00:26:14	INSERT INTO accounts(CustomerID, BranchID, AvailableAmount, AccountType) VALUES (1, 1, 5000.00, 'Check'), (2, 1, 50.00, 'Savings'), (2, 1, 550.0...	13 row(s) affected Records: 13 Duplicates: 0 Warnings: 0
12	00:26:14	UPDATE accounts SET AccountNumber = CONCAT(BranchID, (FLOOR(RAND() * 9000000) + 1000000)) WHERE AccountID >= 100	13 row(s) affected Rows matched: 13 Changed: 13 Warnings: 0
13	00:26:14	UPDATE accounts SET IntrestRate = CASE AccountType WHEN 'Check' THEN 0.95 WHEN 'Savings' THEN 3.00 WHEN 'Loan' THEN 2.50 EN...	13 row(s) affected Rows matched: 13 Changed: 13 Warnings: 0
14	00:26:14	CREATE TABLE transactions(TransactionID int NOT NULL AUTO_INCREMENT, FromAccountID int NOT NULL, ToAccountID int NOT NULL, ...	0 row(s) affected
15	00:26:14	INSERT INTO transactions(FromAccountID, ToAccountID, Amount, TransactionType, TransactionDateTime) VALUES (102, 103, 50.00, 'Transfer', NO...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0
16	00:26:14	CREATE TABLE accounttransactions (TransactionID int NOT NULL, ToAccountID int NOT NULL, FromAccountID int NOT NULL, FOREIG...	0 row(s) affected
17	00:26:14	INSERT IGNORE INTO accounttransactions(TransactionID, ToAccountID, FromAccountID) SELECT t.TransactionID, t.ToAccountID, t.FromAccountID...	6 row(s) affected, 6 warning(s): 1062 Duplicate entry '1-103-102' for key 'accounttransactions.PRIMARY' 1062 Duplicate entry '2-100-112' for key 'accou...

Figure 44: Output from CreateAndPopulateSchema.sql

After running the .SQL script, the result from the MySQL workbench action output window should display all green ticks next to each query, apart from the last one. This is the AccountTransactions INSERT statement, and repeat values are expected when inserting (due to the OR condition), so only one record is inserted. After this has run, there are select statements at the end of the script file. You can uncomment these and see that the data was inserted correctly into the tables.

Reflection:

I think more diligent planning initially could have extended the database further, allowing for more concrete bases for future functionalities. I stuck to the scope of the project well and gave a good starter structure that can be easily added to. If I were to repeat this project, I would do more research and ask more questions before conceptualizing the database structure – doing this would have helped in creating a wider base structure for the database, allowing easier upscaling.

Bibliography

Alam, M. A., Chen, Y. C., & Mansor, N. (2022, Feburary 13). *Predicting savings adequacy using machine learning: A behavioural economics approach*. Retrieved from sciencedirect: https://www.sciencedirect.com/science/article/abs/pii/S0957417422008314

Datacamp. (2023, Feburary). *Decision Tree Classification in Python Tutorial*. Retrieved from datacamp: https://www.datacamp.com/tutorial/decision-tree-classification-python

M'Raihi, D., Machani, S., Diversinet Corp., Pei, M., Symantec, Rydell, J., . . . Verisign, Inc. (2011, May). *TOTP: Time-Based One-Time Password Algorithm*. Retrieved from rfc-editor: https://www.rfc-editor.org/rfc/rfc6238#section-4